

# Sequential Decision Making: Theory and Practice

## Lecture 1: Logistics and Stochastic Bandits

Yuejie Chi

Yale Statistics & Data Science



Fall 2026

# Outline

---

Course introduction

Logistics

Stochastic bandits

# Supervised learning

---

Given training data, make prediction on unseen data:

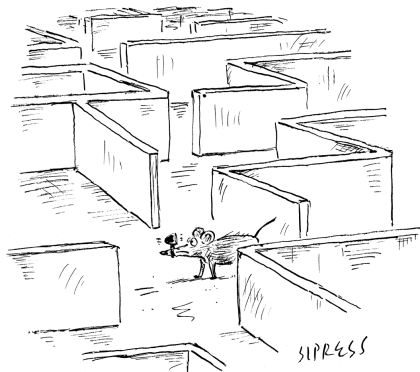


Primarily deal with **pattern recognition**

# Reinforcement learning

In RL, an agent learns by interacting with an environment.

- no training data
- maximize total rewards
- trial-and-error
- sequential and online

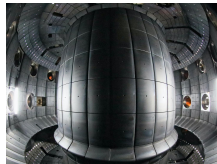
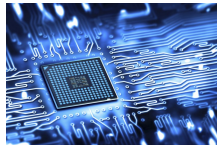


*"Recalculating ... recalculating ..."*

Deal with **decision making**, sometimes with constraints

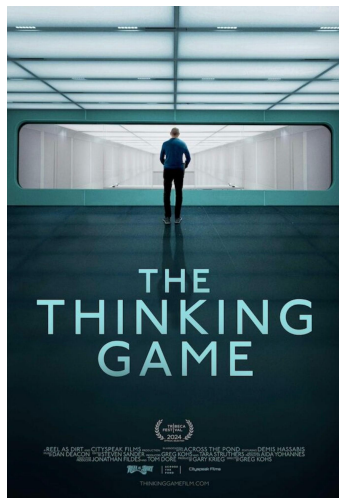


# Recent successes in reinforcement learning (RL)



*RL holds great promise in the next era of artificial intelligence.*

# One more: RL for thinking models

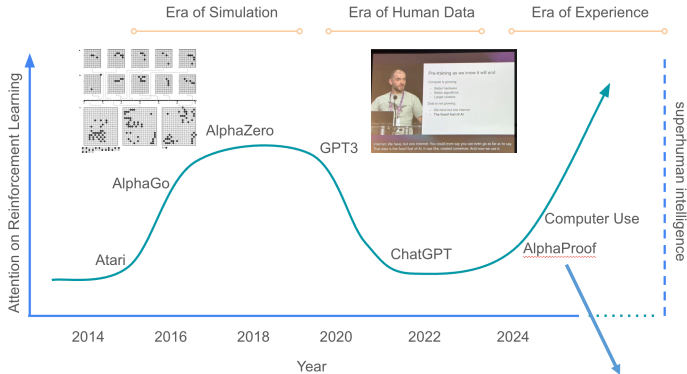


## ***Turing Award Goes to 2 Pioneers of Artificial Intelligence***

Andrew Barto and Richard Sutton developed reinforcement learning, a technique vital to chatbots like ChatGPT.



# The cycles of RL



by human mathematicians, AlphaProof's reinforcement learning (RL) algorithm<sup>1</sup> subsequently generated a hundred million more through continual interaction with a formal proving system. This focus on interactive experience allowed AlphaProof to explore mathematical possibilities beyond the confines of pre-existing formal proofs, so as to discover solutions to novel and challenging problems. Informal mathematics has also achieved success by replacing expert generated data with self-generated data; for example, recent work

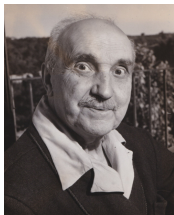
Figure credit: [Silver and Sutton, 2025]

# Sequential decision making

---

*“Those who cannot remember the past are  
condemned to repeat it.”*

—George Santayana



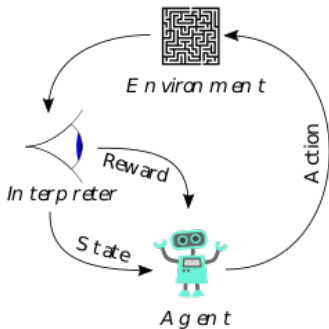
- Games
- Robotics navigation and control
- Pricing and supply chain management
- Recommendation systems
- Portfolio optimization
- Large language models and agents

Learn from past to predict and optimize future performance

# Reinforcement learning

---

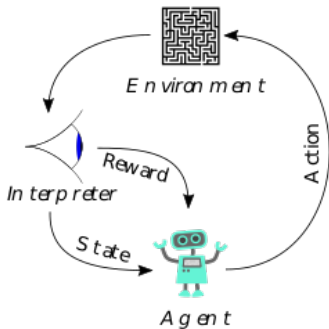
In reinforcement learning (RL), an agent learns through interaction with the (unknown) environment.



# Reinforcement learning

---

In reinforcement learning (RL), an agent learns through interaction with the (unknown) environment.



RL has deep connection with control theory, and is also sometimes called **approximate dynamic programming**. It can be viewed as a type of **optimal control theory** with no pre-defined model of the environment.

# Challenges of RL

---

- explore or exploit: unknown or changing environments
- credit assignment problem: delayed rewards or feedback
- enormous state and action space
- nonconvex optimization

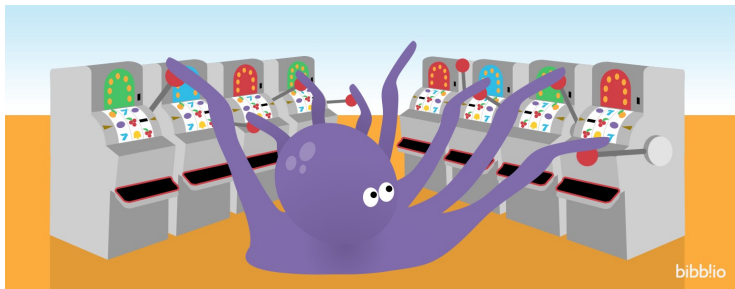




# Multi-arm bandit

---

Which slot machine will give me the most money?



# Learning the best arm

---

Can we **learn** which slot machine gives the most money?



\$1  
\$0  
\$0



\$1  
\$4  
\$0  
\$2  
\$1  
\$3  
\$5



\$1  
\$0  
\$1  
\$2

# Learning the best arm via trial-and-error

---

Which arm do I pick next, so that I maximize my reward over time?



\$1  
\$0  
\$0



\$1  
\$4  
\$0  
\$2  
\$1  
\$3  
\$5



\$1  
\$0  
\$1  
\$2

# Learning the best arm via trial-and-error

---

Which arm do I pick next, so that I maximize my reward over time?



\$1  
\$0  
\$0



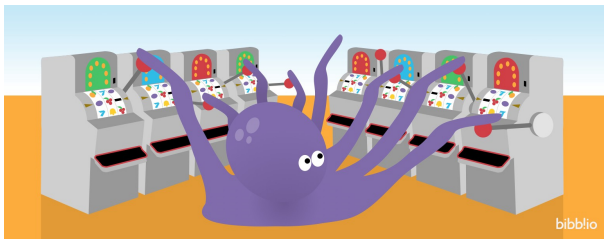
\$1  
\$4  
\$0  
\$2  
\$1  
\$3  
\$5



\$1  
\$0  
\$1  
\$2  
\$12  
\$11

# Exploration-exploitation trade-off

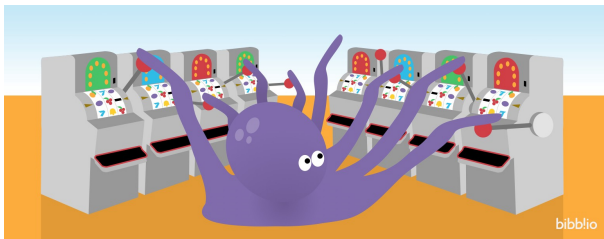
---



Which arm should I play?

# Exploration-exploitation trade-off

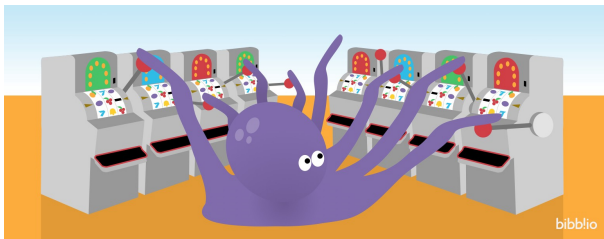
---



Which arm should I play?

- Best arm observed so far? (exploitation)
- Or should I look around to try and find a better arm? (exploration)

# Exploration-exploitation trade-off



Which arm should I play?

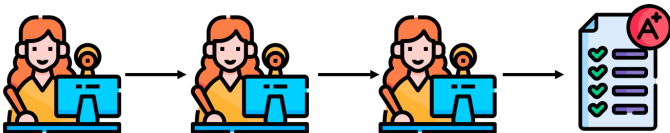
- Best arm observed so far? (exploitation)
- Or should I look around to try and find a better arm? (exploration)

We need both in order to maximize the total reward.

# Credit assignment problem

---

What is the action that leads to the desired outcome?

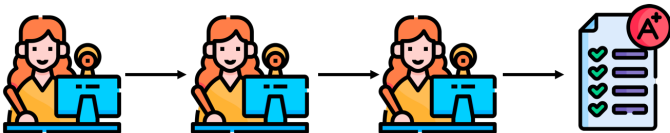




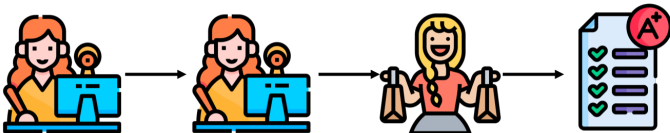
# Credit assignment problem

---

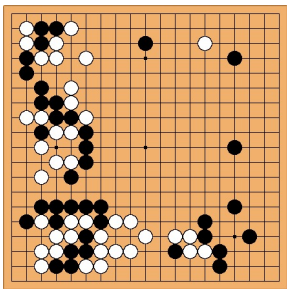
What is the action that leads to the desired outcome?



What if....



# Enormous problem size and function approximation



$$S \approx 2 \cdot 10^{170}$$

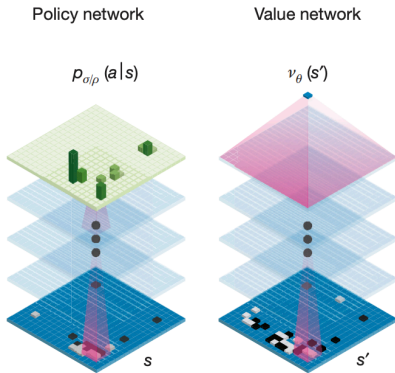


Figure credit: AlphaGo



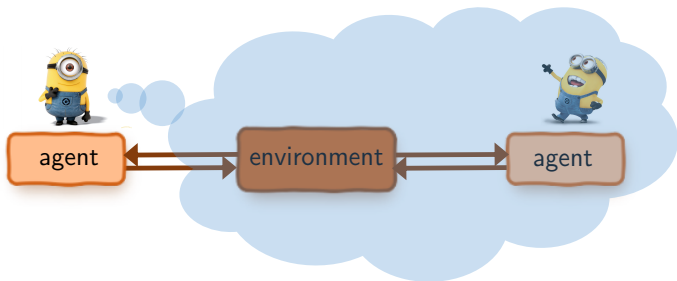
# Challenges in MARL: nonstationarity

---



# Challenges in MARL: nonstationarity

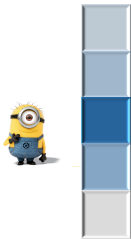
---



From a single-agent perspective:  
the environment is **time-varying** and **nonstationary**!

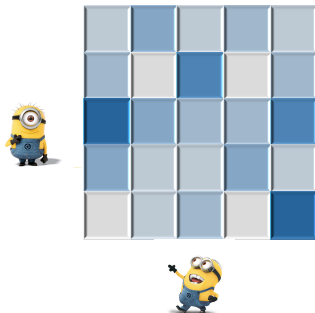
# Challenges in MARL: curse of multiple agents

---



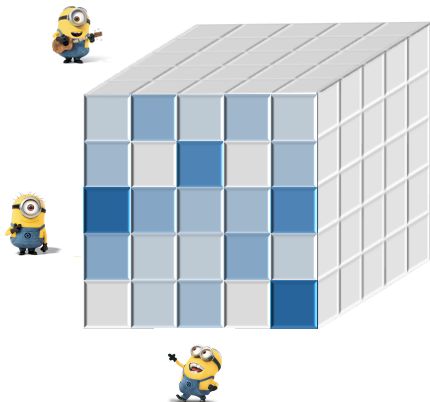
# Challenges in MARL: curse of multiple agents

---



# Challenges in MARL: curse of multiple agents

---



The explosion of choices:  
The joint action space grows **exponentially** with the agents!



# Partial observability in RL

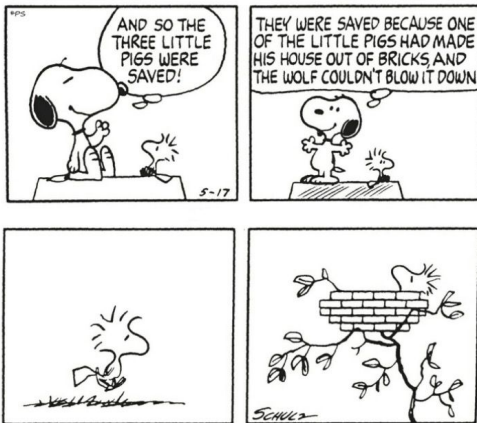
---



# Goal of this course

---

- Build the “foundations” but with an eye on modern applications
- Research-oriented
- models, algorithms and their analyses



# Sample efficiency

---

Collecting data samples might be expensive or time-consuming



clinical trials



autonomous driving



online ads

# Sample efficiency

---

Collecting data samples might be expensive or time-consuming



clinical trials



autonomous driving



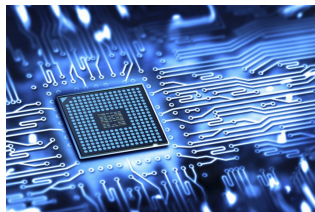
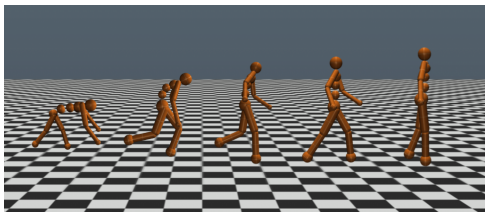
online ads

**Calls for design of sample-efficient RL algorithms!**

# Computational efficiency

---

Running RL algorithms might take a long time and space

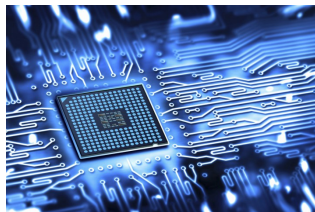
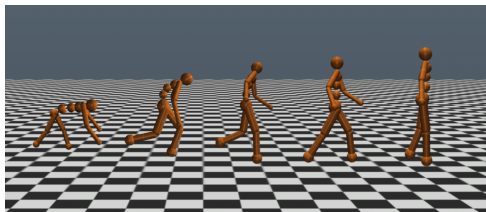


*many CPUs / GPUs / TPUs + computing hours*

# Computational efficiency

---

Running RL algorithms might take a long time and space



*many CPUs / GPUs / TPUs + computing hours*

**Calls for computationally efficient RL algorithms!**

# From asymptotic to non-asymptotic analyses



Non-asymptotic analyses are key to understand sample and computational efficiency in modern RL.

# Logistics



# Basic information

---

- Class: Tuesdays, 9:25 – 11:20 am
- Class location: Watson Center A74
- Instructor: Yuejie Chi, [yuejie.chi@yale.edu](mailto:yuejie.chi@yale.edu)
- Instructor's office hours: Tuesdays, 11:30 am–12:30 pm (right after class)
- Announcements, readings, and logistics: canvas, Ed, and <https://yuejiechi.github.io/sds6190.html>
- Project submissions: Gradescope, unless otherwise noted

# Course format and expectations

---

- Class meets once a week
- Class meetings combine
  - lectures
  - discussion
  - student presentations
- A substantial final project running throughout the semester (more on this later)
- Research-oriented exploration of theoretical foundations and modern applications

# Tentative topics

---

- Stochastic and adversarial bandits
- Markov decision processes and dynamic programming
- Evaluation and planning with a simulator
- Online reinforcement learning
- Imitation learning and offline reinforcement learning
- Policy optimization
- Multi-agent reinforcement learning
- RL for reasoning and alignment of foundation models: RLHF and RLVR
- Cross-cutting themes: sample complexity, computational efficiency, and function approximation

# Useful references

---

We recommend these books and notes, but will not follow them closely ...

- **Reinforcement Learning: Theory and Algorithms**, by Alekh Agarwal, Nan Jiang, Sham M. Kakade, and Wen Sun
- **Reinforcement Learning: An Introduction**, by Richard S. Sutton and Andrew G. Barto
- **Reinforcement Learning and Optimal Control**, by Dimitri P. Bertsekas
- **Bandit Algorithms**, by Tor Lattimore and Csaba Szepesvari
- **Sample Complexity of Reinforcement Learning: A Non-Asymptotic Perspective**, by Yuxin Chen, Yuejie Chi, Jianqing Fan, Gen Li, Yuting Wei, and Yuling Yan

More references will be provided at each lecture.

# Prerequisites

---

- Basic fluency in
  - linear algebra and probability
  - optimization and machine learning
  - programming
- Prior exposure to reinforcement learning is helpful, but not required
- *Concentration inequalities* are a plus, but not necessary
- If you are unsure about your preparation, please ask early

# Grading

---

- Participation and attendance: 5%
- In-class quizzes: 15%
  - ~10 in total, on canvas at the end of each lecture, you can drop 2.
- Midterm presentation: 10%
- Project development: 35%
- Final deliverables and report: 35%

# The project

---

- Pursue an **original research direction** related to modern reinforcement learning and sequential decision making
- Develop the project throughout the semester through
  - proposal drafts and a final proposal
  - research checkpoints
  - peer review and feedback
- Present the project in class
- Submit final deliverables and a report
- Project development (35%) and final deliverables (35%) together account for **70%** of the course grade

# Key project and presentation milestones

---

## September–October

- Sep. 22: project proposal, first draft
- Oct. 6: project proposal, second draft
- Oct. 13: midterm presentation draft
- Oct. 20: midterm presentation and peer feedback
- Oct. 27: project proposal, final version

## November–December

- Nov. 10: project milestone 1
- Nov. 24: project milestone 2
- Dec. 1: final presentation draft
- Dec. 8: final project presentation
- Dec. 15–22: final project report due

*The schedule is provisional and may be updated as the semester proceeds.*



# Rationale of the project schedule

---

- Start early, but allow iterations to refine ideas.
- Read topics before they are “covered”, and come to lectures with questions.
- Require strong maturity to succeed.

# The use of LLMs

---

You are encouraged to use LLMs to help with

- writing and understanding the lecture materials
- brainstorming
- coding

# The use of LLMs

---

You are encouraged to use LLMs to help with

- writing and understanding the lecture materials
- brainstorming
- coding

You own your submitted or posted work and take full responsibility for it.

- Verify the output critically
- Be ready to defend your work when prompted
- Disclose how LLMs were used in submitted work

# Enrollment

---

# Enrollment

---

Enrollment requires faculty approval.

- Students interested in this course should submit a statement of interest, describing their background in relevant courses.
- This should be submitted on Canvas in the “Assignments” area.
- You will be informed within a week after the first meeting of the class.

For the first offering, the target class size is about 20.

Let's get started!

---

Enjoy Yourself



# Stochastic bandits

# A/B testing

---

How do you decide which variation leads to higher traffic/revenue?

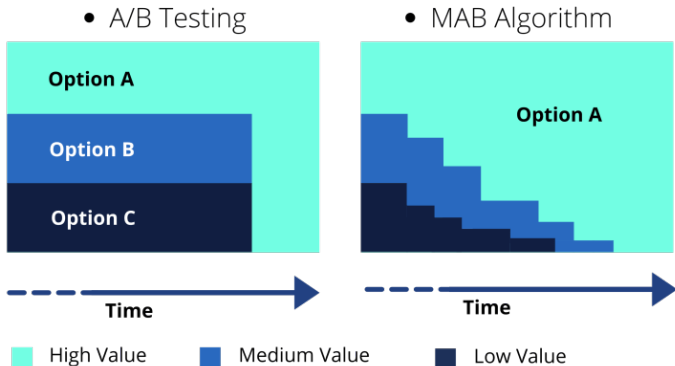


*Figure credit: internet.*

**A/B testing:** explore each variation equally first, then deploy the statistically better one.



# From A/B testing to multi-arm bandits



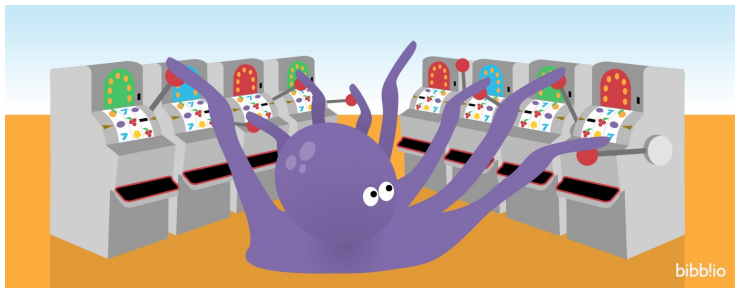
*Figure credit: internet.*

**Multi-arm bandits:** simultaneous exploration and exploitation, dynamic allocation.

# Multi-arm bandit

---

Which slot machine will give me the most money?



First proposed in [Thompson, 1933], popularized by [Robbins, 1952]

# Learning the best arm

---

Can we **learn** which slot machine gives the most money?



\$1  
\$0  
\$0



\$1  
\$4  
\$0  
\$2  
\$1  
\$3  
\$5



\$1  
\$0  
\$1  
\$2

# Formulation

---

We can play multiple rounds  $t = 1, 2, \dots, T$ .

In each round, we **select an arm**  $i_t$  from a fixed set  $i = 1, 2, \dots, n$ ; and **observe the reward**  $r_t$  that the arm gives.

Arm 1



Arm 2



Arm 3



# Formulation

---

We can play multiple rounds  $t = 1, 2, \dots, T$ .

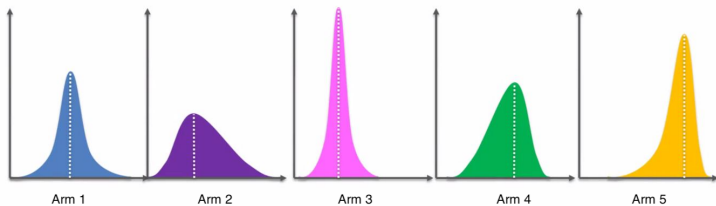
In each round, we **select an arm**  $i_t$  from a fixed set  $i = 1, 2, \dots, n$ ; and **observe the reward**  $r_t$  that the arm gives.



**Objective:** Maximize the total reward over time.

# Stochastic bandit

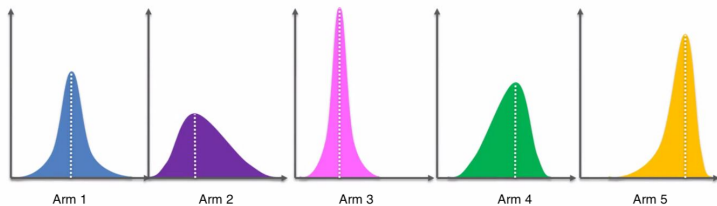
---



- The reward at each arm is **stochastic** (e.g., 1 with probability  $p_i$  and otherwise 0).

# Stochastic bandit

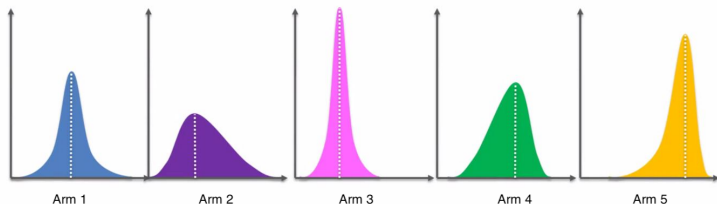
---



- The reward at each arm is **stochastic** (e.g., 1 with probability  $p_i$  and otherwise 0).
- Suppose the rewards are independent over time. The **best arm** is then the arm with highest expected reward.

# Stochastic bandit

---



- The reward at each arm is **stochastic** (e.g., 1 with probability  $p_i$  and otherwise 0).
- Suppose the rewards are independent over time. The **best arm** is then the arm with highest expected reward.

**Example of online ads:** arm = ad, reward = 1 if the user clicks on the ad and 0 otherwise



# Stochastic bandit with i.i.d. rewards

---

We consider a simple setting with i.i.d. bounded rewards.

# Stochastic bandit with i.i.d. rewards

---

We consider a simple setting with i.i.d. bounded rewards.

- Each arm distributes rewards according to some (unknown) distribution over  $[0, 1]$ , with

$$\mathbb{E}[r_{i,t}] = \mu_i, \quad \forall i \in [n], t = 1, 2 \dots$$

- Suppose we play arm  $i_t$  at round  $t$ , and receive the reward

$$r_{i_t,t}$$

drawn i.i.d. from the arm  $i_t$ 's distribution.

# Stochastic bandit with i.i.d. rewards

---

We consider a simple setting with i.i.d. bounded rewards.

- Each arm distributes rewards according to some (unknown) distribution over  $[0, 1]$ , with

$$\mathbb{E}[r_{i,t}] = \mu_i, \quad \forall i \in [n], t = 1, 2 \dots$$

- Suppose we play arm  $i_t$  at round  $t$ , and receive the reward

$$r_{i_t,t}$$

drawn i.i.d. from the arm  $i_t$ 's distribution.

**Partial information:** Every round we cannot observe the reward of all arms: we just know the reward of the arm that we played.

# Regret: performance metric

---

We design algorithms that determine the sequence  $\{i_t\}$ , i.e. *policies*.

**How to evaluate the performance?**

## Definition 1 (Expected regret)

The **expected regret over  $T$  rounds** is defined as

$$R_T = \max_{1 \leq i \leq n} \mathbb{E} \left[ \sum_{t=1}^T (r_{i,t} - r_{i_t,t}) \right] = T\mu^* - \mathbb{E} \left[ \sum_{t=1}^T r_{i_t,t} \right],$$

where  $\mu^* = \max_{1 \leq i \leq n} \mu_i$  is the highest expected reward over all arms.

# Regret: performance metric

---

We design algorithms that determine the sequence  $\{i_t\}$ , i.e. *policies*.

**How to evaluate the performance?**

## Definition 1 (Expected regret)

The **expected regret over  $T$  rounds** is defined as

$$R_T = \max_{1 \leq i \leq n} \mathbb{E} \left[ \sum_{t=1}^T (r_{i,t} - r_{i_t,t}) \right] = T\mu^* - \mathbb{E} \left[ \sum_{t=1}^T r_{i_t,t} \right],$$

where  $\mu^* = \max_{1 \leq i \leq n} \mu_i$  is the highest expected reward over all arms.

- 1st term captures the highest cumulative reward in *hindsight*.

# Regret: performance metric

---

We design algorithms that determine the sequence  $\{i_t\}$ , i.e. *policies*.

**How to evaluate the performance?**

## Definition 1 (Expected regret)

The **expected regret over  $T$  rounds** is defined as

$$R_T = \max_{1 \leq i \leq n} \mathbb{E} \left[ \sum_{t=1}^T (r_{i,t} - r_{i_t,t}) \right] = T\mu^* - \mathbb{E} \left[ \sum_{t=1}^T r_{i_t,t} \right],$$

where  $\mu^* = \max_{1 \leq i \leq n} \mu_i$  is the highest expected reward over all arms.

- 1st term captures the highest cumulative reward in *hindsight*.
- 2nd term captures the *actual* accumulated reward.

# Regret decomposition lemma

---

Since  $\mathbb{E}[r_{i_t,t}] = \mathbb{E}[\sum_{i=1}^n \mu_i \mathbb{I}_{i_t=i}] = \sum_{i=1}^n \mu_i (\mathbb{E} \mathbb{I}_{i_t=i})$ , then

$$\begin{aligned} R_T &= \sum_{t=1}^T \left[ \sum_{i=1}^n \mu^* (\mathbb{E} \mathbb{I}_{i_t=i}) - \sum_{i=1}^n \mu_i (\mathbb{E} \mathbb{I}_{i_t=i}) \right] \\ &= \sum_{i=1}^n \Delta_i \mathbb{E} \left[ \sum_{t=1}^T \mathbb{I}_{i_t=i} \right] \\ &=: \sum_{i=1}^n \Delta_i \mathbb{E} [T_{i,T}] \end{aligned}$$

where

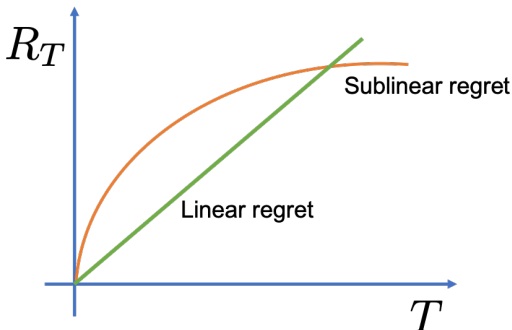
- $\Delta_i = \mu^* - \mu_i$  is the sub-optimality gap of arm  $i$ ;
- $T_{i,T} = \sum_{t=1}^T \mathbb{I}_{i_t=i}$  is the number of times arm  $i$  is played in  $T$  rounds.

# Sublinear regret

---

**Sublinear regret:** most MAB algorithms aim to achieve **sublinear regret**, so that the average regret goes to 0 as  $T \rightarrow \infty$ :

$$\lim_{T \rightarrow \infty} \frac{R_T}{T} = 0$$





## From $\epsilon$ -greedy to UCB algorithm

# Learning the best arm via trial-and-error

---

Which arm do I pick next, so that I maximize my reward over time?



\$1

\$0

\$0



\$1

\$4

\$0

\$2

\$1

\$3

\$5



\$1

\$0

\$1

\$2

# Learning the best arm via trial-and-error

---

Which arm do I pick next, so that I maximize my reward over time?



\$1  
\$0  
\$0



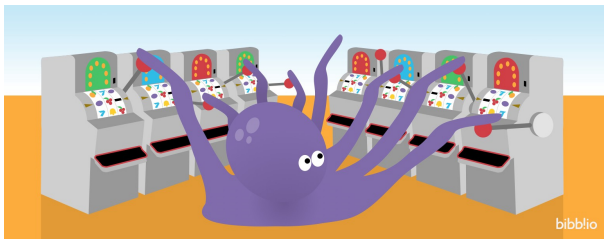
\$1  
\$4  
\$0  
\$2  
\$1  
\$3  
\$5



\$1  
\$0  
\$1  
\$2  
\$12  
\$11

# Exploration-exploitation trade-off

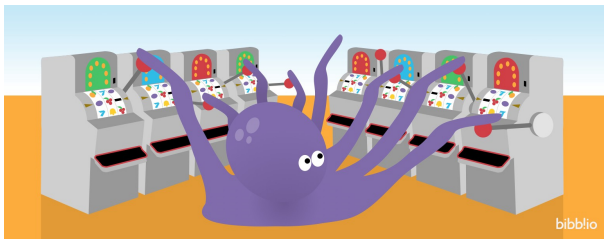
---



Which arm should I play?

# Exploration-exploitation trade-off

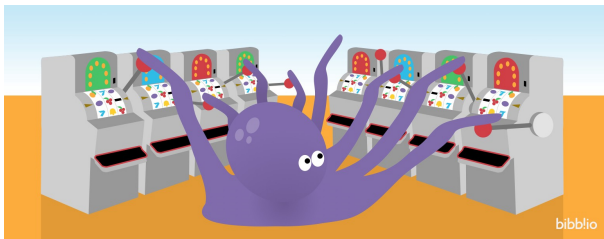
---



Which arm should I play?

- Best arm observed so far? (exploitation)
- Or should I look around to try and find a better arm? (exploration)

# Exploration-exploitation trade-off



Which arm should I play?

- Best arm observed so far? (exploitation)
- Or should I look around to try and find a better arm? (exploration)

We need both in order to maximize the total reward.

# An $\epsilon$ -greedy approach

---

Exploit, but explore a random arm  $\epsilon$  fraction of the time.

- 1 **Initial phase:** Try each arm and observe the reward.

# An $\epsilon$ -greedy approach

---

Exploit, but **explore a random arm  $\epsilon$  fraction of the time.**

- 1 **Initial phase:** Try each arm and observe the reward.
- 2 For each round  $t = n + 1, \dots, T$ :



# An $\epsilon$ -greedy approach

---

Exploit, but **explore a random arm  $\epsilon$  fraction of the time.**

- ① **Initial phase:** Try each arm and observe the reward.
- ② For each round  $t = n + 1, \dots, T$ :
  - Calculate the empirical average reward for each arm  $i$ :

$$\bar{\mu}_{i,t} = \frac{\text{total reward from pulling this arm in the past}}{\text{number of times I pulled this arm}} = \frac{\sum_{t:i_t=i} r_t}{\sum_{t:i_t=i} 1},$$

where  $i_t$  is the index of the arm played at time  $t$ ,  $r_t$  is the reward.

# An $\epsilon$ -greedy approach

---

Exploit, but **explore a random arm  $\epsilon$  fraction of the time.**

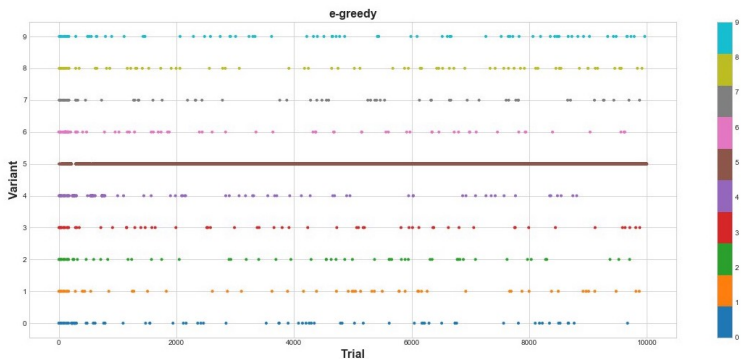
- ① **Initial phase:** Try each arm and observe the reward.
- ② For each round  $t = n + 1, \dots, T$ :
  - Calculate the empirical average reward for each arm  $i$ :

$$\bar{\mu}_{i,t} = \frac{\text{total reward from pulling this arm in the past}}{\text{number of times I pulled this arm}} = \frac{\sum_{t:i_t=i} r_t}{\sum_{t:i_t=i} 1},$$

where  $i_t$  is the index of the arm played at time  $t$ ,  $r_t$  is the reward.

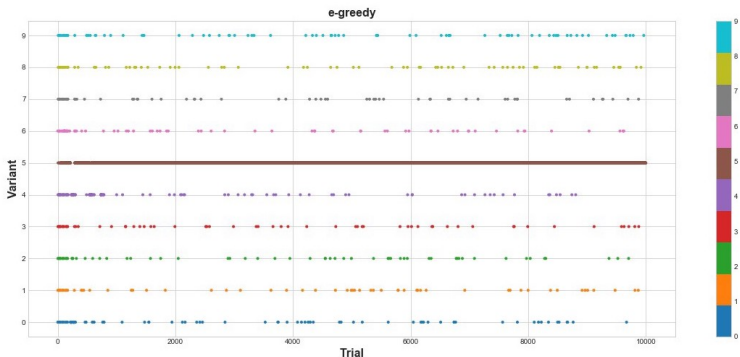
- With probability  $1 - \epsilon$ , play the arm with highest  $\bar{\mu}_{i,t}$  and observe the reward. Otherwise, choose an arm at random and observe the reward.

# Understanding $\epsilon$ -greedy



- In the first thousand iterations, all arms are chosen fairly frequently.

# Understanding $\epsilon$ -greedy



- In the first thousand iterations, all arms are chosen fairly frequently.
- Eventually the algorithm realizes that arm 5 has the highest expected reward.

# Regrets of greedy policies

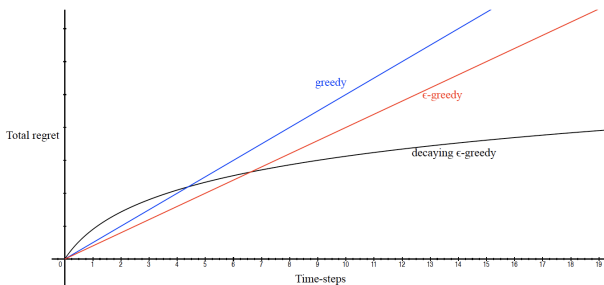


Figure credit: David Silver's lecture.

- Greedy policy incurs linear regret since it can lock on a sub-optimal policy.

# Regrets of greedy policies

---

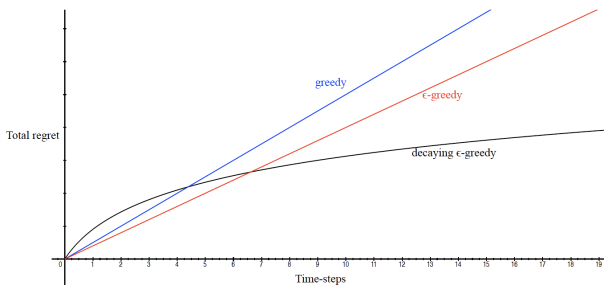


Figure credit: David Silver's lecture.

- Greedy policy incurs linear regret since it can lock on a sub-optimal policy.
- $\epsilon$ -greedy always explores by  $\epsilon$  fraction and therefore its regret is still linear (recall the regret decomposition lemma).

# Regrets of greedy policies

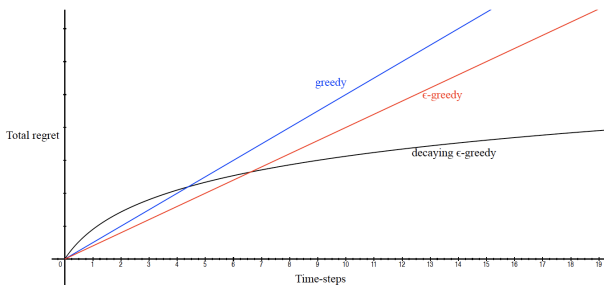


Figure credit: David Silver's lecture.

- Greedy policy incurs linear regret since it can lock on a sub-optimal policy.
- $\epsilon$ -greedy always explores by  $\epsilon$  fraction and therefore its regret is still linear (recall the regret decomposition lemma).
- Decaying  $\epsilon$  helps, however it is hard to design the schedule.

# The UCB algorithm

---

[Auer et al., 2002]: the idea is to **always try the best arm**, where “best” includes exploration and exploitation.

- ➊ **Initial phase:** try each arm and observe the reward.



# The UCB algorithm

---

[Auer et al., 2002]: the idea is to **always try the best arm**, where “best” includes exploration and exploitation.

- 1 **Initial phase:** try each arm and observe the reward.
- 2 For each round  $t = n + 1, \dots, T$ :

# The UCB algorithm

---

[Auer et al., 2002]: the idea is to **always try the best arm**, where “best” includes exploration and exploitation.

- 1 **Initial phase:** try each arm and observe the reward.
- 2 For each round  $t = n + 1, \dots, T$ :
  - Calculate the **UCB (upper confidence bound) index** for each arm  $i$ :

$$\text{UCB}_{i,t} = \bar{\mu}_{i,t} + \sqrt{\frac{\log t}{T_{i,t}}},$$

where  $\bar{\mu}_{i,t}$  is the empirical average reward for arm  $i$  and  $T_{i,t}$  is the number of times arm  $i$  has been played up to round  $t$ .

# The UCB algorithm

---

[Auer et al., 2002]: the idea is to **always try the best arm**, where “best” includes exploration and exploitation.

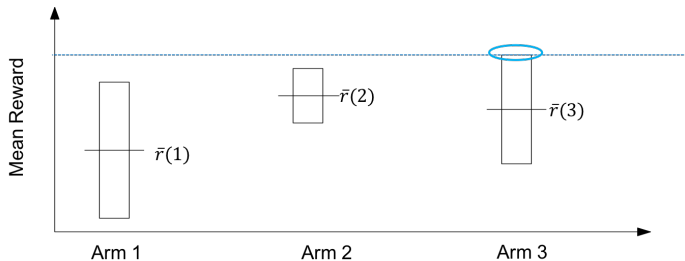
- 1 **Initial phase:** try each arm and observe the reward.
- 2 For each round  $t = n + 1, \dots, T$ :
  - Calculate the **UCB (upper confidence bound) index** for each arm  $i$ :

$$\text{UCB}_{i,t} = \bar{\mu}_{i,t} + \sqrt{\frac{\log t}{T_{i,t}}},$$

where  $\bar{\mu}_{i,t}$  is the empirical average reward for arm  $i$  and  $T_{i,t}$  is the number of times arm  $i$  has been played up to round  $t$ .

- Play the arm with the highest UCB index and observe the reward.

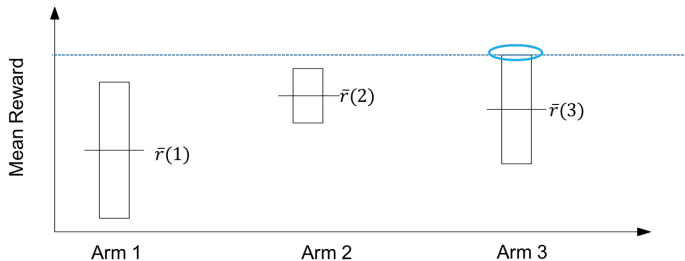
# Understanding UCB



$$\text{UCB}_{i,t} = \bar{\mu}_{i,t} + \sqrt{\frac{\log t}{T_{i,t}}},$$

- **Exploitation:**  $\bar{\mu}_{i,t}$  is the average observed reward. High observed rewards of an arm leads to high UCB index.

# Understanding UCB



$$UCB_{i,t} = \bar{\mu}_{i,t} + \sqrt{\frac{\log t}{T_{i,t}}},$$

- **Exploitation:**  $\bar{\mu}_{i,t}$  is the average observed reward. High observed rewards of an arm leads to high UCB index.
- **Exploration:**  $\sqrt{\frac{\log t}{T_{i,t}}}$  decreases as we make more observations ( $T_{i,t}$  grows). Few observations of an arm leads to high UCB index.

# Theory of UCB algorithm

---

## Theorem 2 (Instance-dependent regret bound of UCB)

For  $T \geq n$ , the expected regret of UCB algorithm is upper bounded as

$$R_T \leq \sum_{i:\Delta_i>0} \left( \frac{4\log T}{\Delta_i} + 8\Delta_i \right) \leq \sum_{i:\Delta_i>0} \frac{4\log T}{\Delta_i} + 8n,$$

where  $\Delta_i = \mu^* - \mu_i$  is the sub-optimality gap of arm  $i$ .

# Theory of UCB algorithm

## Theorem 2 (Instance-dependent regret bound of UCB)

For  $T \geq n$ , the expected regret of UCB algorithm is upper bounded as

$$R_T \leq \sum_{i: \Delta_i > 0} \left( \frac{4 \log T}{\Delta_i} + 8 \Delta_i \right) \leq \sum_{i: \Delta_i > 0} \frac{4 \log T}{\Delta_i} + 8n,$$

where  $\Delta_i = \mu^* - \mu_i$  is the sub-optimality gap of arm  $i$ .

- The regret bound scales with the *harmonic mean* of the gaps,

$$R_T \lesssim \frac{n \log T}{\text{harmonic mean}(\{\Delta_i\})}.$$

- E.g.  $\Delta_2 = \frac{1}{2}$ ,  $\Delta_3 = \frac{1}{2}$ , harmonic mean =  $\frac{1}{2}$ .
- E.g.  $\Delta_2 = \frac{1}{10}$ ,  $\Delta_3 = \frac{1}{2}$ , harmonic mean =  $\frac{1}{6}$ .

# Theory of UCB algorithm

## Theorem 2 (Instance-dependent regret bound of UCB)

For  $T \geq n$ , the expected regret of UCB algorithm is upper bounded as

$$R_T \leq \sum_{i: \Delta_i > 0} \left( \frac{4 \log T}{\Delta_i} + 8 \Delta_i \right) \leq \sum_{i: \Delta_i > 0} \frac{4 \log T}{\Delta_i} + 8n,$$

where  $\Delta_i = \mu^* - \mu_i$  is the sub-optimality gap of arm  $i$ .

- The regret bound scales with the *harmonic mean* of the gaps,

$$R_T \lesssim \frac{n \log T}{\text{harmonic mean}(\{\Delta_i\})}.$$

- E.g.  $\Delta_2 = \frac{1}{2}$ ,  $\Delta_3 = \frac{1}{2}$ , harmonic mean =  $\frac{1}{2}$ .
- E.g.  $\Delta_2 = \frac{1}{10}$ ,  $\Delta_3 = \frac{1}{2}$ , harmonic mean =  $\frac{1}{6}$ .
- When  $\Delta_i$ 's are constants, the regret scales as (ignoring  $n$ )

$$R_T = O(\log T),$$

which is nearly the best we can hope for! (We'll see why later.)



## Gap-free bound of UCB algorithm

---

The gap-dependent bound may become too loose when  $\Delta_i$  is, say, asymptotically small,  $\Delta_i \sim \log T/T$ .

# Gap-free bound of UCB algorithm

---

The gap-dependent bound may become too loose when  $\Delta_i$  is, say, asymptotically small,  $\Delta_i \sim \log T/T$ .

Fortunately, this can be fixed by studying the following **instance-independent (aka worst-case) bound**.

# Gap-free bound of UCB algorithm

---

The gap-dependent bound may become too loose when  $\Delta_i$  is, say, asymptotically small,  $\Delta_i \sim \log T/T$ .

Fortunately, this can be fixed by studying the following **instance-independent (aka worst-case) bound**.

## Theorem 3 (Instance-independent regret bound of UCB)

*For  $T \geq n$ , the expected regret of UCB algorithm is upper bounded as*

$$R_T \leq 4\sqrt{nT \log T} + 8n.$$

# Gap-free bound of UCB algorithm

---

The gap-dependent bound may become too loose when  $\Delta_i$  is, say, asymptotically small,  $\Delta_i \sim \log T/T$ .

Fortunately, this can be fixed by studying the following **instance-independent (aka worst-case) bound**.

## Theorem 3 (Instance-independent regret bound of UCB)

*For  $T \geq n$ , the expected regret of UCB algorithm is upper bounded as*

$$R_T \leq 4\sqrt{nT \log T} + 8n.$$

- When  $n = O(1)$ , the regret scales as

$$R_T = O(\sqrt{T \log T}) = \tilde{O}(\sqrt{T})$$

- The logarithmic factor can be shaved away [Audibert and Bubeck, 2009].

# **Analysis**

# Toolkit: Hoeffding's inequality

## Theorem 4 (Hoeffding's inequality)

Let  $X_1, X_2, \dots, X_n$  be independent random variables satisfying  $a_i \leq X_i \leq b_i$ . Then for all  $\delta \geq 0$ ,

$$\mathbb{P} \left( \left| \sum_{i=1}^n X_i - \mathbb{E} \left[ \sum_{i=1}^n X_i \right] \right| \geq \varepsilon \right) \leq 2 \exp \left( - \frac{2\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right).$$

# Toolkit: Hoeffding's inequality

## Theorem 4 (Hoeffding's inequality)

Let  $X_1, X_2, \dots, X_n$  be independent random variables satisfying  $a_i \leq X_i \leq b_i$ . Then for all  $\delta \geq 0$ ,

$$\mathbb{P} \left( \left| \sum_{i=1}^n X_i - \mathbb{E} \left[ \sum_{i=1}^n X_i \right] \right| \geq \varepsilon \right) \leq 2 \exp \left( - \frac{2\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right).$$

Setting  $a_i = 0$ ,  $b_i = 1$ , and  $\mathbb{E}[X_i] = \mu$ , we obtain

$$\begin{aligned} \mathbb{P} \left( \left| \frac{1}{n} \sum_{i=1}^n X_i - \mu \right| \geq \varepsilon \right) &\leq 2 \exp(-2n\varepsilon^2) \\ \Rightarrow \left| \frac{1}{n} \sum_{i=1}^n X_i - \mu \right| &\leq \sqrt{\frac{\log(2/\delta)}{2n}} \quad \text{with prob. } 1 - \delta. \end{aligned}$$

# Toolkit: Hoeffding's inequality

## Theorem 4 (Hoeffding's inequality)

Let  $X_1, X_2, \dots, X_n$  be independent random variables satisfying  $a_i \leq X_i \leq b_i$ . Then for all  $\delta \geq 0$ ,

$$\mathbb{P} \left( \left| \sum_{i=1}^n X_i - \mathbb{E} \left[ \sum_{i=1}^n X_i \right] \right| \geq \varepsilon \right) \leq 2 \exp \left( - \frac{2\varepsilon^2}{\sum_{i=1}^n (b_i - a_i)^2} \right).$$

Setting  $a_i = 0$ ,  $b_i = 1$ , and  $\mathbb{E}[X_i] = \mu$ , we obtain

$$\begin{aligned} \mathbb{P} \left( \left| \frac{1}{n} \sum_{i=1}^n X_i - \mu \right| \geq \varepsilon \right) &\leq 2 \exp(-2n\varepsilon^2) \\ \Rightarrow \left| \frac{1}{n} \sum_{i=1}^n X_i - \mu \right| &\leq \sqrt{\frac{\log(2/\delta)}{2n}} \quad \text{with prob. } 1 - \delta. \end{aligned}$$

This will allow us to talk about how the mean reward concentrates around the true mean.



# Implications of Hoeffding's inequality

---

For each arm  $i$  at time  $t$ , with probability at least  $1 - 2/t^2$ ,

$$|\bar{\mu}_{i,t} - \mu_i| < \sqrt{\frac{\log t}{T_{i,t}}}$$
$$\Rightarrow \text{UCB}_{i,t} = \bar{\mu}_{i,t} + \sqrt{\frac{\log t}{T_{i,t}}} \geq \mu_i.$$

## **Optimism in the face of uncertainty:**

acting according to the UCB index, which is an upper bound of the true mean  $\mu_i$ .



# Bound the number of sub-optimal pulls

---

Recall that

$$R_T = \sum_{i=1}^n \Delta_i \underbrace{\mathbb{E}[T_{i,T}]}_{\text{control target}}.$$

# Bound the number of sub-optimal pulls

---

Recall that

$$R_T = \sum_{i=1}^n \Delta_i \underbrace{\mathbb{E}[T_{i,T}]}_{\text{control target}}.$$

**Key observation:** at each  $t$ , the UCB index of the sub-optimal arms  $i \neq i^*$  will be sufficiently apart from the optimal one and arm  $i$  will not get pulled (i.e.  $i_{t+1} \neq i$ ), as long as  $T_{i,t}$  is sufficiently large:

$$\begin{aligned} \text{UCB}_{i,t} &= \bar{\mu}_{i,t} + \sqrt{\frac{\log t}{T_{i,t}}} \leq \mu_i + 2\sqrt{\frac{\log t}{T_{i,t}}} && \text{(Hoeffding)} \\ &\leq \mu_{i^*} \leq \text{UCB}_{i^*,t} && \text{(optimism/Hoeffding)} \end{aligned}$$

as long as

$$T_{i,t} \geq \frac{4 \log t}{\Delta_i^2}$$

with probability at least  $1 - 4/t^2$  (we applied Hoeffding twice).

## Bound the number of sub-optimal pulls

---

$$\begin{aligned}\mathbb{E}[T_{i,T}] &\leq \mathbb{E}\left[\sum_{t=0}^{T-1} \mathbb{I}(i_{t+1} = i)\right] \\&= \mathbb{E}\left[\sum_{t=0}^{T-1} \mathbb{I}\left(i_{t+1} = i, T_{i,t} < \frac{4 \log t}{\Delta_i^2}\right)\right] + \mathbb{E}\left[\sum_{t=0}^{T-1} \mathbb{I}\left(i_{t+1} = i, T_{i,t} \geq \frac{4 \log t}{\Delta_i^2}\right)\right] \\&\leq \frac{4 \log T}{\Delta_i^2} + \sum_{t=n}^{T-1} \mathbb{P}\left(i_{t+1} = i, T_{i,t} \geq \frac{4 \log t}{\Delta_i^2}\right) \\&\leq \frac{4 \log T}{\Delta_i^2} + \sum_{t=n}^{T-1} \mathbb{P}\left(i_{t+1} = i | T_{i,t} \geq \frac{4 \log t}{\Delta_i^2}\right) \mathbb{P}\left(T_{i,t} \geq \frac{4 \log t}{\Delta_i^2}\right) \\&\leq \frac{4 \log T}{\Delta_i^2} + \sum_{t=n}^{T-1} \frac{4}{t^2} \\&\leq \frac{4 \log T}{\Delta_i^2} + 8.\end{aligned}$$

## A key lemma

---

### Lemma 5 (bounding the number of pulls of sub-optimal arms)

*For any arm with  $\Delta_i > 0$ , it holds that*

$$\mathbb{E}[T_{i,T}] \leq \frac{4 \log T}{\Delta_i^2} + 8.$$

## A key lemma

---

### Lemma 5 (bounding the number of pulls of sub-optimal arms)

For any arm with  $\Delta_i > 0$ , it holds that

$$\mathbb{E}[T_{i,T}] \leq \frac{4 \log T}{\Delta_i^2} + 8.$$

**Proof of Theorem 2:**

$$\begin{aligned} R_T &= \sum_{i=1}^n \Delta_i \mathbb{E}[T_{i,T}] \leq \sum_{\Delta_i > 0} \Delta_i \left( \frac{4 \log T}{\Delta_i^2} + 8 \right) \\ &= \sum_{\Delta_i > 0} \left( \frac{4 \log T}{\Delta_i} + 8 \Delta_i \right). \end{aligned}$$

# From gap-dependent to gap-independent bounds

---

**Intuition:** for some  $\Delta$  to be determined later,

- For arms  $\{i : \Delta_i \geq \Delta\}$  with **large gaps**: use the gap-dependent bound

$$\mathbb{E}[T_{i,T}] \leq \frac{4 \log T}{\Delta_i^2} + 8;$$

- For arms  $\{i : \Delta_i < \Delta\}$  with **small gaps**: use the naive bound

$$\sum_i \mathbb{E}[T_{i,T}] \leq T.$$

# From gap-dependent to gap-independent bounds

---

**Intuition:** for some  $\Delta$  to be determined later,

- For arms  $\{i : \Delta_i \geq \Delta\}$  with **large gaps**: use the gap-dependent bound

$$\mathbb{E}[T_{i,T}] \leq \frac{4 \log T}{\Delta_i^2} + 8;$$

- For arms  $\{i : \Delta_i < \Delta\}$  with **small gaps**: use the naive bound

$$\sum_i \mathbb{E}[T_{i,T}] \leq T.$$

Hence,

$$\begin{aligned} R_T &= \sum_{i=1}^n \Delta_i \mathbb{E}[T_{i,T}] \leq \sum_{i: \Delta_i \geq \Delta} \Delta_i \left( \frac{4 \log T}{\Delta_i^2} + 8 \right) + \sum_{i: \Delta_i < \Delta} \Delta_i \mathbb{E}[T_{i,T}] \\ &\leq \frac{4n \log T}{\Delta} + 8n + \Delta T. \end{aligned}$$



# From gap-dependent to gap-independent bounds

**Intuition:** for some  $\Delta$  to be determined later,

- For arms  $\{i : \Delta_i \geq \Delta\}$  with **large gaps**: use the gap-dependent bound

$$\mathbb{E}[T_{i,T}] \leq \frac{4 \log T}{\Delta_i^2} + 8;$$

- For arms  $\{i : \Delta_i < \Delta\}$  with **small gaps**: use the naive bound

$$\sum_i \mathbb{E}[T_{i,T}] \leq T.$$

Hence,

$$\begin{aligned} R_T &= \sum_{i=1}^n \Delta_i \mathbb{E}[T_{i,T}] \leq \sum_{i: \Delta_i \geq \Delta} \Delta_i \left( \frac{4 \log T}{\Delta_i^2} + 8 \right) + \sum_{i: \Delta_i < \Delta} \Delta_i \mathbb{E}[T_{i,T}] \\ &\leq \frac{4n \log T}{\Delta} + 8n + \Delta T. \end{aligned}$$

Choosing  $\Delta = \sqrt{\frac{4n \log T}{T}}$ , we obtain  $R_T \leq 4\sqrt{nT \log T} + 8n$ .

# Wrap-up

---

Quiz 1: on canvas and submit why you are eager to take this class!

Due date: this Friday.

# References I

---



Audibert, J.-Y. and Bubeck, S. (2009).  
Minimax policies for adversarial and stochastic bandits.  
*In COLT*, pages 217–226.



Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002).  
Finite-time analysis of the multiarmed bandit problem.  
*Machine learning*, 47(2):235–256.



Robbins, H. (1952).  
Some aspects of the sequential design of experiments.  
*Bulletin of the American Mathematical Society*, 58(5):527–535.



Silver, D. and Sutton, R. S. (2025).  
Welcome to the era of experience.  
*Google AI*, 1:11.



Thompson, W. R. (1933).  
On the likelihood that one unknown probability exceeds another in view of the evidence of two samples.  
*Biometrika*, 25(3-4):285–294.